

Task1

The screenshot shows the Amazon DynamoDB Clusters console. The left sidebar contains navigation links for DynamoDB (Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, Settings) and DAX (Clusters, Subnet groups, Parameter groups, Events). The main content area is titled 'Clusters (1)' and includes a search bar and buttons for 'View details', 'Delete', and 'Create cluster'. A table lists the cluster details:

Cluster name	Status	Node type	Nodes	Availability Zones	Endpoint
mycluster	Available	dax.r7i.large	3	ap-south-1a	daxc:/myclusterj2ecat.dax-clusters.ap-south-1.amazonaws.com

The screenshot shows the AWS VPC console for a VPC named 'project-vpc' (ID: vpc-0a2c949cbd09848c2). The left sidebar shows navigation links for VPC dashboard, AWS Global View, and Virtual private cloud (Your VPCs, Subnets, Route tables, Internet gateways, Egress-only internet gateways, DHCP option sets, Elastic IPs, Managed prefix lists, NAT gateways, Peering connections). The main content area displays the VPC details in a grid:

Details Info			
VPC ID vpc-0a2c949cbd09848c2	State Available	Block Public Access Off	DNS hostnames Enabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-0d332718531add6a7	Main route table rtb-07372cc4d049f5503
Main network ACL acl-00dfa7b6a2f3e426f	Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -
IPv6 CIDR (Network border group) -	Network Address Usage metrics Disabled	Route 53 Resolver DNS Firewall rule groups -	Owner ID 119922150537

Below the details grid are tabs for 'Resource map', 'CIDRs', 'Flow logs', 'Tags', and 'Integrations'. The 'Resource map' tab is selected, showing a 'Resource map Info' section with a 'Show all details' toggle.

Task2

```
package org.example;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dax.DaxClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.GetItemResponse;
import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;
import java.net.URI;
import java.util.HashMap;
import java.util.Map;
import java.util.stream.DoubleStream;
public class DAXDemo {
    public static void main(String[] args) {
        //get this end point created using AWS cloud
        // create a cluster...
        String daxEndpoint = "dax://daxcluster01.ee31f0.dax-clusters.ap-south-1.amazonaws.com";

        // we are creating DAX client
        DoubleStream DaxClient;
        DaxClient daxClient = DaxClient.builder()
            .endpointOverride(URI.create(daxEndpoint))
            .region(Region.AP_SOUTH_1)
            .credentialsProvider(DefaultCredentialsProvider.create())
```

```
public class DAXDemo {
    public static void main(String[] args) {
        .region(Region.AP_SOUTH_1)
        .credentialsProvider(DefaultCredentialsProvider.create())
        .build();

        String tableName = "Employees01";
        String KeyName = "ID";
        String KeyValue = "10001";
        Map<String, AttributeValue> item = new HashMap<>();
        item.put(KeyName, AttributeValue.builder().s(KeyValue).build());
        item.put("msg", AttributeValue.builder().s("we are creating DAX ").build());
        PutItemRequest request = PutItemRequest.builder()
            .tableName(tableName)
            .item(item)
            .build();
        daxClient.putItem(request);
        Map<String, AttributeValue> getItem = new HashMap<>();
        getItem.put(KeyName, AttributeValue.builder().build());
        GetItemRequest request = GetItemRequest.builder()
            .tableName(tableName)
            .key(getItem)
            .build();
        GetItemResponse response = daxClient.getItem(request);
        System.out.println("Dax working ..");
```